

Projet Génie Logiciel : Puissance 4 3D

Léontine Hordoir, Rym Zerzouh, Émilien Manceau, Nancy Le Corre

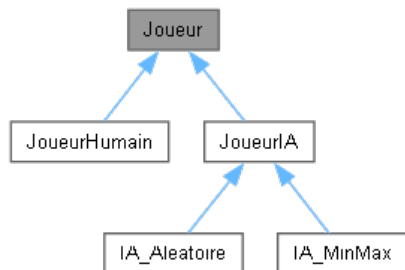
Polytech Lyon MAM

2 Juin 2026

- 1 Architecture globale
- 2 Modélisation de l'espace (Plateau 3D)
- 3 L'Intelligence artificielle (MinMax)
- 4 Communication réseau (MPI)
- 5 Validation et tests
- 6 Documentation
- 7 Conclusion

Une structure Orientée Objet

- Classe abstraite `Joueur` imposant la méthode `choisirCoup()` pour tous.
- Déclinaisons en `JoueurHumain` et `JoueurIA`.
- Polymorphisme pour intégrer `IA_Aleatoire` et `IA_MinMax`.



Un tableau 1D pour représenter la 3D

Pour optimiser les accès mémoire cruciaux lors du parcours de l'arbre MinMax, la grille n'est pas un tableau tridimensionnel lourd, mais un `std::array` 1D ultra-rapide de 80 cases.

```
1 int Plateau3D::index(int p_longueur, int p_largeur, int p_hauteur) const {
2     // Calcul mathématique de l'index unique
3     int index = p_largeur + (p_longueur * largeur) + (p_hauteur * largeur * longueur);
4
5     if (index > (largeur * hauteur * longueur)){
6         std::cout << "Erreur, out of range" << std::endl;
7         return 0;
8     }
9     return index;
10 }
11 }
```

Listing 1 – Aplatissement des coordonnées 3D en 1D

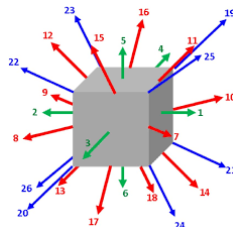
Les 13 directions de l'espace

- 3 Axes (Horizontal, Vertical, Profondeur).
- 6 Diagonales de faces.
- 4 Diagonales spatiales pures.

Le programme génère au démarrage (via `initialiser_lignes()`) un catalogue des lignes gagnantes pour accélérer l'évaluation.

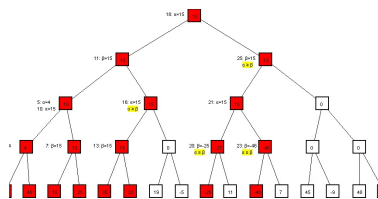
```
1 static constexpr Vec3 directions[] = {  
2     {1,0,0}, {0,1,0}, {0,0,1}, // Axes  
3     {1,1,0}, {1,-1,0}, {1,0,1}, {1,0,-1},  
4     {0,1,1}, {0,1,-1}, // Diagonales faces  
5     {1,1,1}, {1,1,-1},  
6     {1,-1,1}, {1,-1,-1} // Diag. spatiales  
7 };  
8
```

Listing 2 – Vecteurs des directions 3D



Anticipation et Optimisation

- **Profondeur 4** : Le meilleur compromis temps/performance.
- **Élagage** $\alpha - \beta$: Coupe les branches d'exploration inutiles.
- **Ordre de parcours** : Priorisation du centre via ordreX et ordreY.



Heuristique : La fonction d'évaluation

Objectif : détecter les menaces

La fonction `evaluerPlateau` analyse chaque ligne de 4 cases. Elle détecte les victoires et repère aussi les **doubles menaces (fourchettes)** :

```
1 // Detection des lignes de 3 pions
2 if (nbrIA == 3 && nbrAdv == 0) menacesIA++;
3 if (nbrAdv == 3 && nbrIA == 0) menacesAdv++;
4
5 // Bonus et Malus massifs pour les fourchettes (2 lignes de 3)
6 if (menacesIA >= 2) {
7     scoreTotal += 50000;
8 }
9 if (menacesAdv >= 2) {
10     scoreTotal -= 80000; // Force l'IA a bloquer l'adversaire
11 }
12
```

Listing 3 – Logique d'évaluation heuristique

Vérification

Avant même de lancer une lourde récursion MinMax, choisirCoup vérifie s'il existe une victoire immédiate ou une défaite immédiate à bloquer, évitant ainsi un mauvais choix lié à l'horizon de recherche.

```
1 // Je bloque l'adversaire s'il menace de gagner au prochain tour
2 for (int x : ordreX) {
3     for (int y : ordreY) {
4         if (plateauSimule.est_coup_valide(x, y)) {
5             int z = plateauSimule.ajouter_pion(x, y, couleurAdverse);
6             if (plateauSimule.verifier_victoire(x, y, z, couleurAdverse)) {
7                 plateauSimule.retirer_pion(x, y);
8                 return {x, y}; // Je pose mon pion ici en urgence
9             }
10            plateauSimule.retirer_pion(x, y);
11        }
12    }
13 }
14
```

Listing 4 – Extrait : Blocage immédiat de l'adversaire

Le pont entre le jeu local et l'arbitre distant

La classe JoueurMPI enveloppe notre IA et gère les communications. Elle traduit nos coups locaux en positions pour le réseau :

```
1 void JoueurMPI::calculerEtEnvoyer(int rankArbitre) {  
2     // L'IA calcule le meilleur coup localement  
3     std::pair<int, int> monChoix = monIA->choisirCoup(plateau);  
4     plateau.ajouter_pion(monChoix.first, monChoix.second, monIA->getCouleur());  
5  
6     // Traduction et envoi sur le reseau  
7     position monCoupPourLArbitre;  
8     monCoupPourLArbitre.seth(monCoupPourLArbitre.getalpha(monChoix.first));  
9     monCoupPourLArbitre.setv(monChoix.second);  
10    sendCoup(monCoupPourLArbitre, rankArbitre);  
11 }  
12
```

Listing 5 – Extrait de la méthode calculerEtEnvoyer

Valider l'algorithme par les statistiques

Pour prouver l'efficacité de nos choix sans saturer la console, la classe Partie intègre un mode lancer_silencieux. Ce mode génère des centaines de matchs en arrière-plan.

```
1 Joueur* Partie::lancer_silencieux(bool j1commence) {  
2     bool gagne = false;  
3     Joueur* joueurActuel = j1commence ? this->J1 : this->J2;  
4  
5     while (gagne == false && this->tours_joues < max_tours) {  
6         std::pair<int, int> xy = joueurActuel->choisirCoup(this->plateau);  
7         int z = this->plateau.ajouter_pion(xy.first, xy.second, joueurActuel->getCouleur());  
8         gagne = this->plateau.verifier_victoire(xy.first, xy.second, z, joueurActuel->getCouleur());  
9         // ... (Alternance du joueur)  
10    }  
11    return gagne ? joueurActuel : nullptr;  
12 }  
13 }
```

Listing 6 – Simulation automatisée

Scénario 1 : Intelligence vs Hasard

MinMax (Profondeur 4) affronte une IA Aléatoire. L'alternance du joueur de départ garantit l'équité.

Résultats sur 10 parties

MinMax Expert : 10 victoires

IA Aléatoire : 0 victoire

Scénario 2 : Le Poids de la Profondeur

MinMax affronte son propre code, mais avec des horizons de prédiction différents (4 coups vs 2 coups).

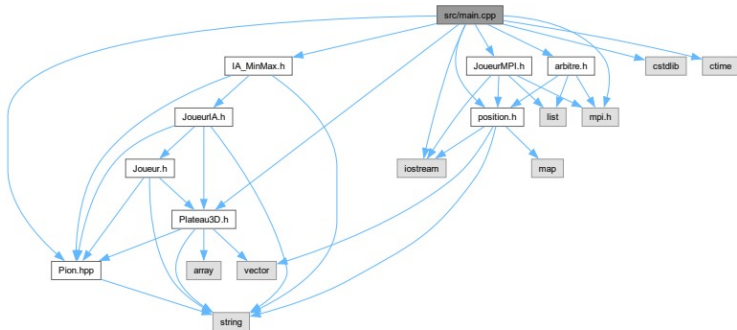
Résultats sur 10 parties

MinMax Profondeur 4 : Victoire majoritaire

MinMax Profondeur 2 : Défaite ou Nul

Génération automatique à partir du code source

Graphe des dépendances par inclusion de main.cpp:

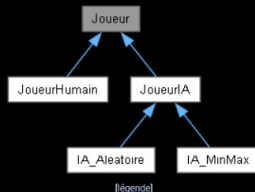


My Project

Classe abstraite représentant un joueur. Plus de détails...

```
#include <Joueur.h>
```

Graphes d'héritage de Joueur:



Fonctions membres publiques

Joueur (std::string nom, Pion couleur)

Joueur (Pion couleur)

Joueur (const **Joueur** &other)

Joueur & **operator=** (const **Joueur** &other)

virtual std::pair< int, int > **choisirCoup** (const **Plateau3D** &plateau)=0

Demande au joueur de choisir un coup.

std::string **getNom** () const

Retourne le nom du joueur.

Pion **getCouleur** () const

Merci de votre attention !